# Hong Kong Metropolitan University

# COMP8090SEF

# Data Structures and Algorithms

# Course Project Report

# Submission date: **14/04/2026**

I declare that:

(i) I have read and checked that all parts of the project (including proposal, code programs, and reports), they are contributed by me, here submitted is original except for source material explicitly acknowledged;

(ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;

(iii) I am aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;

(iv) I confirm that I have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and

(v) I am aware that I should be held responsible and liable to disciplinary actions, irrespective of whether I have signed the declaration and whether I have contributed, directly or indirectly, to the problematic contents.

I confirm that I have read through and understood the project requirements. I understand that failure to comply with the project requirements will result in score deduction.

| **Name** | **SID** |
|---|---|
| **WONG Yan-ho, Michael** | **14194311** |

# Table of Content

## Introduction and System Overview

In this task, it will show an analysis of the Library Borrowing System that developed by a Python-based application. Then it demonstrate the principle through object-oriented programming, also show how to implement a console-based library management solution that handle some general function such as book inventory, member registration, and loan tracking operation. The application architecture follow a layer design pattern, separate concern across model, repository, service and user interface component. It can enhance maintainability, testability and scalability while showcasing industry-standard software design.

## Core Function and Technical Feature

### *Book Management*

The system provide book inventory management capability through the implementation of the book class and repository layer. The core function allow administrator to register new book with unique identifier, title, author and tracking multiple copies. It can provide real-time monitoring of each book availability by display the number of available copies in relation to the total copies. Also the system can manage copy count automatically by adjusting the number of available copies during borrow and return procedure in order to ensure stability. Additionally, it can list all books along with their status. The Book class employ encapsulation via properties and incorporate validation logic to prevent inconsistent states, such as returning more copies than have been borrowed.

### *Member Management*

The member management function will facilitate the tracking of library patron and their borrowing privilege. It will include member registration, allow the creation of member profile with unique ID, name and define the borrowing limit. The borrowing limit are configurable, with a default maximum of seven books per member. The system monitor active loan for each member through internal list of book ID. On the other hand, it will perform eligibility validation by automatically checking a member borrow quota before approve any loan record. The member class extend the base person class, demonstrate the principle of inheritance while incorporating library-specific functionality.

### *Loan Operation*

The loan management sub-system form the core business logic of the application. It will handle different essential function such as book borrowing, return processing, due date management, overdue detection and maintaining loan history record etc. When user request to borrow a book, the system process the request by validate the user eligibility and the availability of the book. Then upon return, the sub-system manage the book return process procedure, it

will automatically restore the book copy status and update the loan record accordingly.

The system also manage the due date by calculate and tracking them based on loan period, with a default duration of 14 days. It include built-in logic function to identify overdue loan by compare current date with due date. The system will maintain comprehensive record of all borrowing transaction, tracking their status throughout their cycle. The Loan class will serve as a transaction record that links member and book, containing temporal data and status flag to accurately represent loan detail.

***User Interface***

The ConsoleUI class offer a text-based console interface that facilitate interactive access to system. It provide navigation, present user with number option for use. The interface include input validation and error handling to ensure that user command are correctly process and manage invalid input. Also it will provide format output display for present information about book, member and active loan record in clear and organized manner. The system was equipped with exception handling mechanism that capture error and present user-friendly error message.

## Self-Reflection on Weakness and Future improvement

Despite the system functional for basic library operation, several weakness point have been identified. First of all, the current implementation rely on in-memory data storage using python dictionary and list, meaning all data is lost when the application terminates. Integrating a persistent storage solution such as SQLite or JSON-based file storage would address it. Secondly, the system lack of user authentication and role-based access control, such as different kind of user has full administrative permission to control. Addressing to this problem, it will use a login system with distinct role, restricting sensitive operation based on the user role. Then, the search functionality is limited to exact ID lookup with no support for title, author, or keyword search. Implement string match or index would improve the usability. Finally, the system does not handle concurrent access, running multiple instance simultaneously could cause data inconsistency. A client-server architecture with thread-safe operations would resolve this.

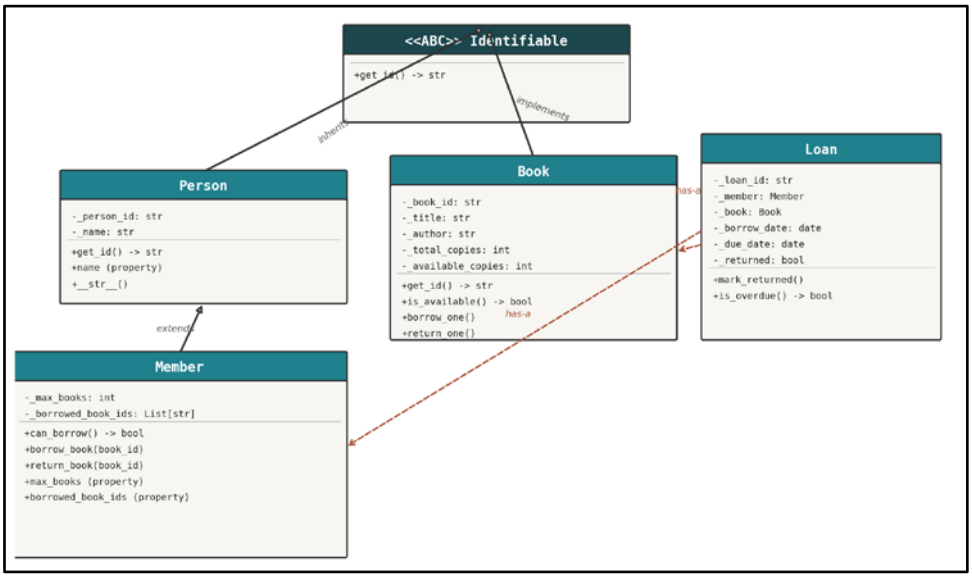
## Conclusion

The system demonstrate core object-oriented principle and software engineering practice through a practical application. It also show the OOP concept that is abstraction, encapsulation, inheritance, polymorphism, and composition while maintaining a clear separation of concern across architectural layer. Although limited in persistence, concurrency, and user management, it provide a solid foundation for future enhancement. The modular architecture facilitate extensibility and incremental improvement without full rewrite.

## Appendix

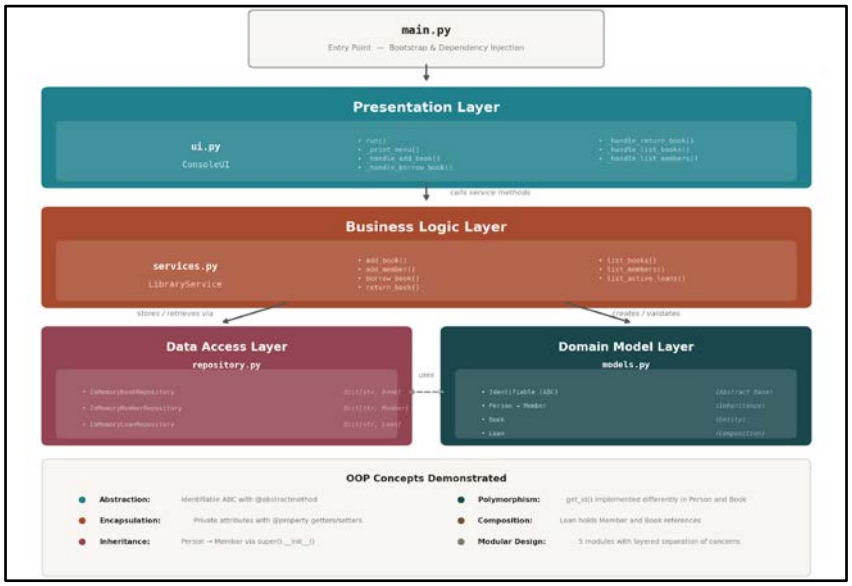
In this section, it will contain the architectural diagram, class relationship, and process flowcharts referenced in the main report.

### *Class Inheritance and Relationship*



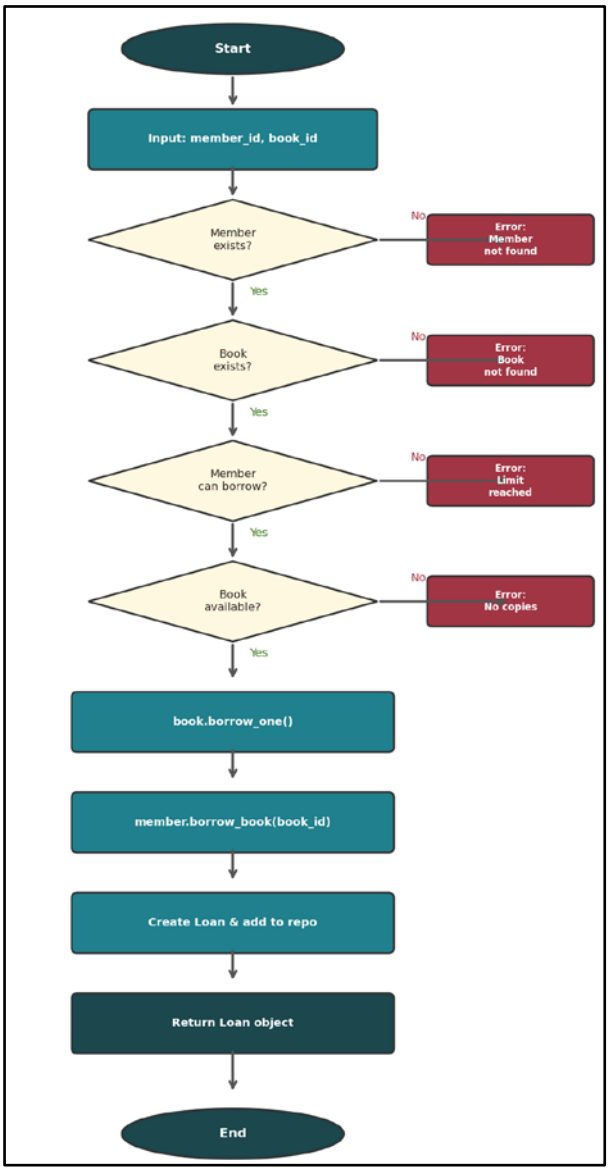

In this diagram, it illustrate the class hierarchy of the Library Borrowing System. The abstract base class (ABC) define the get_id() interface, which is implemented by both the person and book class. The member class extend person through inheritance, adding library-specific attribute and method. The loan class will demonstrate composition by holding references to both Member and Book object.

### *Layered System Architecture*



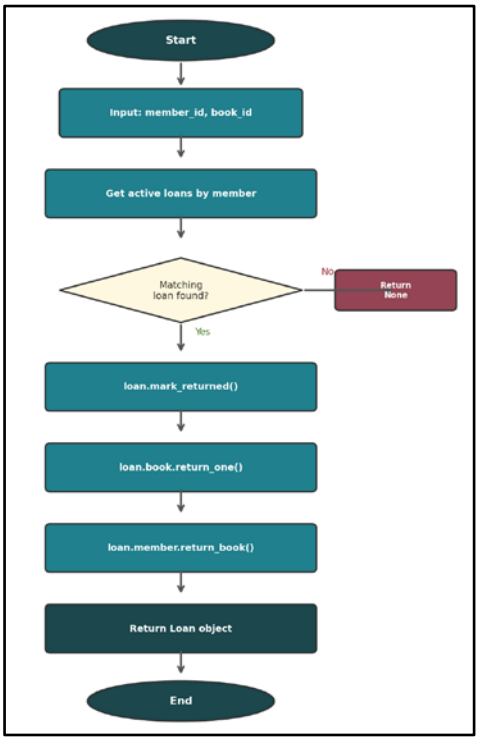
This diagram are describe four-layer architecture. The presentation layer (ui.py) handle user interaction. The business logic layer (services.py) execute rule and validation. The data access layer (repository.py) manage in-memory storage. The domain model layer (models.py) define entity class. The main.py will serve as the bootstrapping entry point that merge all layer together.

***Borrow Book Process flow chart***



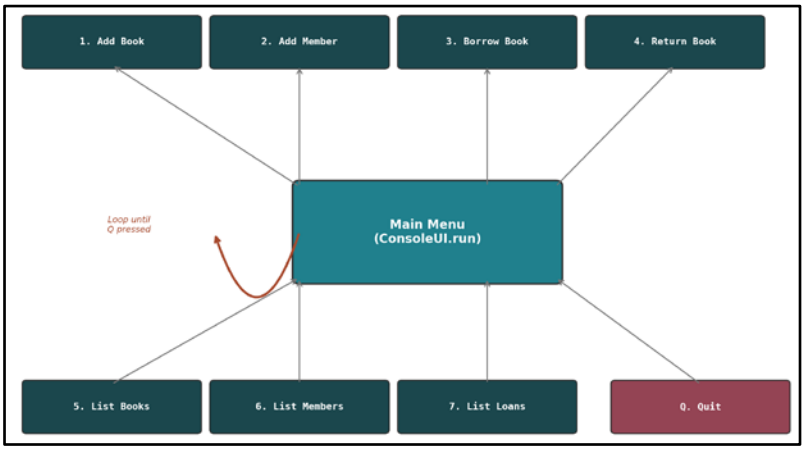

In this flow chart, it show the chain for the borrow_book operation in library_service. The process perform four sequential checking procedure: (a) member existence, (b) book existence, (c) member borrowing eligibility, and (d) book copy availability. Only when all four-checking pass the system proceed to decrement the available copies, loan record and update the member borrow list.

### *Return Book Process flow chart*




This flow chart illustrate the return_book process. The system search for an active loan matching the given member_id and book_id. If the data was found, it mark the loan as returned, restore the book available copy count, and remove the book from the member borrow list. If no matching loan record was found, the method then return Nil.

### *Console UI Menu Navigation Flow*




In this figure, it show the menu-driven navigation of the ConsoleUI. The main loop display seven number operation plus a quit function. Each selection dispatch to a dedicated handler method. The loop will continue until the user enter command "Q".

***OOP concept mapping***

In this table, it will show what kind of OOP concept required to concrete implementation in the project.

| OOP Concept | Implementation | Description |
|---|---|---|
| Abstraction | models.py — identifiable (ABC) | Abstract base class with @abstractmethod get_id(); forces subclasses to provide identity |
| Encapsulation | models.py — All entity classes | Private attribute (_name, _book_id, etc.) with @property getter/setter; borrowed_book_ids returns a copy to prevent external mutation |
| Inheritance | models.py — Person → Member | Member inherit from Person, reusing _person_id and name; super().__init__() chains constructor |
| Polymorphism | models.py — identifiable.get_id() | Person, Book both implement get_id(); can be treated uniformly through the identifiable interface. |
| Composition | models.py — Loan class | Loan hold reference to Member and Book object (has-a relationship); Loan cannot exist without them |
| Modular Design | All 5 module | Separated into main.py, models.py, repository.py, services.py, ui.py — clear separation of concern across layer |

***Module Responsibility***

In this table, it conclude each python module, layer in the architecture and key class.

| Module | Layer | Key Class |
|---|---|---|
| main.py | Entry Point | Nil |
| models.py | Domain Model | Identifiable, Person, Member, Book, Loan |
| repository.py | Data Access | InMemoryBookRepo, InMemoryMemberRepo, InMemoryLoanRepo |
| services.py | Business Logic | LibraryService |
| ui.py | Presentation | ConsoleUI |

***Class Method Summary***

It provide a list of all class, method, parameter, return type and purpose description.

| Class | Method | Parameter | Return | Description |
|---|---|---|---|---|
| identifiable | get_id() | Nil | str | Abstract method; returns unique identifier |
| person | init_() | person_id, name | Nil | Initialize with ID and name (encapsulated) |
| person | get_id() | Nil | str | Return _person_id |
| person | name (getter) | Nil | str | Property getter for name |
| person | name (setter) | new_name: str | Nil | Property setter for name |
| member | init__() | member_id, name, max_books=5 | Nil | Calls super().__init__(); sets borrowing limit |
| member | can_borrow() | Nil | Boolean | True if borrowed < max_books |
| member | borrow_book() | book_id: str | Nil | Append book_id to borrowed list |
| member | return_book() | book_id: str | Nil | Remove book_id from borrowed list |
| book | init_() | book_id, title, author, total_copies=1 | Nil | Initialize with all book attributes |
| book | is_available() | Nil | Boolean | True if available_copies > 0 |
| book | borrow_one() | Nil | Nil | Decrement available; raises ValueError if 0 |
| book | return_one() | Nil | Nil | Increments available; raises ValueError if full |
| loan | init__() | loan_id, member, book, loan_days=14 | Nil | Set date; _returned = False |
| loan | mark_returned() | Nil | Nil | Set returned to True |
| loan | is_overdue() | Nil | Boolean | True if not returned and past due date |
| libraryservice | add_book() | book_id, title, author, total_copies | Nil | Validates uniqueness; create book |
| libraryservice | add_member() | member_id, name, max_books | Nil | Validates uniqueness; create member |
| libraryservice | borrow_book() | member_id, book_id, loan_days | Loan | 4-step validation; create Loan |
| libraryservice | return_book() | member_id, book_id | Loan\|None | Find active loan; processes return |

***Data Structure Usage***

This table show different kind of python data structure will be used in the project and the specific role.

| Data Structure | Python Type | Location | Purpose |
|---|---|---|---|
| dictionary | Dict[str, Book] | inMemoryBookRepository._books | lookup/insert by book_id |
| dictionary | Dict[str, Member] | inMemoryMemberRepository._members | lookup/insert by member_id |
| dictionary | Dict[str, Loan] | inMemoryLoanRepository._loans | lookup/insert by loan_id |
| list | List[str] | Member._borrowed_book_ids | Tracks active borrow per member |
| list (filtered) | List[Loan] | InMemoryLoanRepository.list_active_loans() | List comprehension filter for active loan |
| integer | int | LibraryService._next_loan_id | Auto-increment counter for loan ID |
| date | datetime.date | Loan._borrow_date, Loan._due_date | Temporal tracking and overdue detection |

***Project Demonstration with Test Case***

This section will show all test case executed against the Library Borrowing System.

***Normal Operation Test Case***

| TC-01 | Add Book to System |
|---|---|
| **Prerequisite** | System initialized with empty repository |
| **Input** | add_book("B001", "Clean Code", "Robert C. Martin", 3)<br>add_book("B002", "Introduction to Algorithms", "Cormen et al.", 2)<br>add_book("B003", "Design Patterns", "Gang of Four", 1)<br>add_book("B004", "The Pragmatic Programmer", "Hunt & Thomas", 2) |
| **Output** | Book(B001, Clean Code, Robert C. Martin, available=3/3) Book(B002, Introduction to Algorithms, Cormen et al., available=2/2) Book(B003, Design Patterns, Gang of Four, available=1/1) Book(B004, The Pragmatic Programmer, Hunt & Thomas, available=2/2) |

| TC-02 | Add Member to System |
|---|---|
| **Prerequisite** | System initialized with empty member repository |
| **Input** | add_member("M001", "Alice", 3) add_member("M002", "Bob", 2)<br>add_member("M003", "Charlie", 5) |
| **Output** | Member(M001, Alice, borrowed=0)<br>Member(M002, Bob, borrowed=0)<br>Member(M003, Charlie, borrowed=0) |

| TC-03 | Borrow Book |
|---|---|
| **Prerequisite** | B001 (3 copies), B002 (2 copies) in system; M001 (max=3), M002 (max=2) registered |
| **Input** | borrow_book("M001", "B001") → Alice borrows Clean Code<br>borrow_book("M001", "B002") → Alice borrows Intro to Algorithms<br>borrow_book("M002", "B001") → Bob borrows Clean Code |
| **Output** | Loan(L0001, member=M001, book=B001, due=2026-04-13, active)<br>Loan(L0002, member=M001, book=B002, due=2026-04-13, active)<br>Loan(L0003, member=M002, book=B001, due=2026-04-13, active) |

| TC-04 | List Active Loan |
|---|---|
| **Prerequisite** | 3 active loans exist from TC-03 |
| **Input** | list_active_loans() |
| **Output** | Loan(L0001, member=M001, book=B001, due=2026-04-13, active)<br>Loan(L0002, member=M001, book=B002, due=2026-04-13, active)<br>Loan(L0003, member=M002, book=B001, due=2026-04-13, active) |

| TC-05 | Book Availability after borrowing |
|---|---|
| **Prerequisite** | B001 borrowed twice (of 3 copies), B002 borrowed once (of 2 copies) |
| **Input** | list_books() |
| **Output** | Book(B001, Clean Code, Robert C. Martin, available=1/3)<br>Book(B002, Introduction to Algorithms, Cormen et al., available=1/2)<br>Book(B003, Design Patterns, Gang of Four, available=1/1)<br>Book(B004, The Pragmatic Programmer, Hunt & Thomas, available=2/2) |

| TC-06 | Return book successfully |
|---|---|
| **Prerequisite** | M001 has active loan L0001 for B001 |
| **Input** | return_book("M001", "B001") |
| **Output** | Returned: Loan(L0001, member=M001, book=B001, due=2026-04-13, returned) After return: Book(B001, available=2/3) Active loans reduced from 3 to 2 |

***Error Handling Test Case***

| TC-07 | Duplicate book ID rejection |
|---|---|
| **Prerequisite** | B001 already exists in the system |
| **Input** | add_book("B001", "Duplicate", "Author", 1) |
| **Output** | Error caught: Book ID already exists |

| TC-08 | Duplicate member ID rejection |
|---|---|
| **Prerequisite** | M001 already exists in the system |
| **Input** | add_member("M001", "Duplicate Member", 3) |
| **Output** | Error caught: Member ID already exists |

| TC-09 | Member borrowing limit enforcement |
|---|---|
| **Prerequisite** | M002 (max=2) already has 2 active loans |
| **Input** | borrow_book("M002", "B004") |
| **Output** | Error caught: Member has reached maximum borrowing limit |

| TC-10 | No available copies |
|---|---|
| **Prerequisite** | B003 has 1 total copy, already borrowed by M002 |
| **Input** | borrow_book("M003", "B003") |
| **Output** | Error caught: Book has no available copies |

| TC-11 | Non existent member |
|---|---|
| **Prerequisite** | No member with ID "M999" exists |
| **Input** | borrow_book("M999", "B001") |
| **Output** | Error caught: Member not found |

| TC-12 | Non existent book |
|---|---|
| **Prerequisite** | No book with ID "B999" exists |
| **Input** | borrow_book("M001", "B999") |
| **Output** | Error caught: Book not found |

| TC-13 | Return non-existing loan |
|---|---|
| **Prerequisite** | M003 has no active loan for B001 |
| **Input** | return_book("M003", "B001") |
| **Output** | Return result for non-existing loan: None |

| TC-14 | Member status after mixed operation |
|---|---|
| **Prerequisite** | M001 borrowed 2, returned 1; M002 borrowed 2; M003 borrowed 0 |
| **Input** | list_members() |
| **Output** | Member(M001, Alice, borrowed=1)<br>Member(M002, Bob, borrowed=2)<br>Member(M003, Charlie, borrowed=0) |

| TC-15 | Overdue detection logic |
|---|---|
| **Prerequisite** | New loan created with default 14-day period (today: 2026-03-30) |
| **Input** | Create loan → check is_overdue() → backdate to 30 days ago → check again |
| **Output** | Due date: 2026-04-13 → is_overdue(): False Backdated due date: 2026-03-14 → is_overdue(): True |

| TC-16 | Final System State Verification |
|---|---|
| **Prerequisite** | After all 15 test cases executed |
| **Input** | list_books(), list_members(), list_active_loans() |
| **Output** | Books: B001(2/3), B002(1/2), B003(0/1), B004(2/2) Members: Alice(1), Bob(2), Charlie(0) Active: L0002(M001-B002), L0003(M002-B001), L0004(M002-B003) |

**GitHub Repository Link**

- https://github.com/s1419431/COMP8090SEF-Project-Task-1

**Video Presentation**

- https://shorturl.at/V5aPD

**Presentation Slide**

- https://shorturl.at/7Czfv